

Paper 04 - Boundary Repair

CQE-paper-04

CQECMPLX Formal Paper Series

Review-facing PDF built 2026-06-12

Construction Basis

Define boundary repair as the transport operation that converts failed joins into typed constraints for the next legal route.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- promoted formal paper: production\formal-papers\CQE-paper-04\FORMAL_PAPER.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-04\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-04\verify_boundary_repair.py
- receipt: production\formal-papers\CQE-paper-04\boundary_repair_receipt.json (Typed boundary repair: pass)
- body: production\papers\CQE-paper-04\PAPER-BODY.md
- source: production\papers\CQE-paper-04\SOURCE.md
- formal: production\papers\CQE-paper-04\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-04\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-04\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-04\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-04.md

Paper 04 - Boundary Repair

Abstract

Paper 04 formalizes boundary repair as the step that turns a failed or nonclosing join into a typed constraint for the next legal route. The paper does not claim that every failure is repaired merely by naming it. A repair exists only when the failed join is converted into a replayable constraint with enough information for the next transport to act.

The construction consumes the preceding papers:

Paper 01: LCR supplies the local state.
 Paper 02: correction supplies typed residue.
 Paper 03: axis/sheet supplies a second coordinate system.
 Paper 04: boundary repair turns residue + coordinate into a next-route constraint.

Proof/Exposure Hierarchy

The proof-carrying content of this paper is the mathematics: the definitions, lemmas, constructions, examples, and receipts that establish the claimed result. Paper 00, hand routes, analog tools, workbook language, and obligation ledgers are supplemental validation and exposure layers. They exist to make the math inspectable, reproducible, and accessible without requiring a particular software stack. The hand route is not the purpose of the paper; it is a way to expose the same state transitions with ordinary marks, tokens, lines, or any equivalent physical substitute.

Definitions

An **LCR state** is $s = (L, C, R)$ in $\{0, 1\}^3$.

A **correction residue** is a row from Paper 02 where

$\text{corr}(L, C, R) = C$ and not $R = 1$.

A **local coordinate** is the Paper 03 axis/sheet coordinate:

$\text{coord}(s) = (\text{axis}(s), \text{sheet}(s))$.

A **failed join** is a correction residue that lacks a legal next route.

A **boundary repair constraint** is a record with at least these fields:

```
state
axis_sheet
reason
status
next_legal_routes
source_paper
target_paper
```

The status is `constraint`, not `proof`.

Main Claim

Theorem 4.1, Typed Boundary Repair. A failed join is repairable in the CQECMPLX paper kernel if and only if it can be converted into a typed constraint that preserves the original state, the Paper 03 coordinate, the reason for failure, and at least one next legal route.

Proof

If the failed join is not recorded with its state, coordinate, reason, and next legal route, the next transport has no reproducible object to consume. The failure may be observed, but it is not repaired.

If those fields are present, the next transport receives a determinate constraint. It can accept, defer, or reject that constraint with a receipt. Thus the join has been repaired at the boundary level: not by becoming a theorem, but by becoming a legal input to the next route.

The construction is idempotent. Applying the repair operation to an already typed constraint returns the same constraint, because the state, coordinate, reason, and next route are already fixed. QED.

Concrete Boundary From Papers 02 and 03

Paper 02 gives two correction states:

```
(0,1,0)
(1,1,0)
```

Paper 03 gives their coordinates:

```
(0,1,0) -> (axis 2, sheet 0)
(1,1,0) -> (axis 3, sheet 1)
```

Paper 04 converts each into a constraint row:

```
state: original LCR state
axis_sheet: Paper 03 coordinate
reason: Paper 02 correction fired
status: constraint
next_legal_routes: Paper 05 path-carrier intake, Paper 03 stronger theorem intake
```

This is the first explicit example of a failed local join becoming usable future work without being prematurely promoted to proof.

Falsifier

The falsifier for Paper 04 is an untyped failure:

```
{"status": "failed"}
```

This is not a repair. It lacks state, coordinate, reason, and next route. The verifier must reject it.

Hand Reconstruction

- Start with a Paper 02 correction row.
- Copy the LCR state.
- Look up the Paper 03 axis/sheet coordinate.
- Write the reason: `correction fired`.
- Mark the status as `constraint`.
- Write at least one next legal route.
- Verify that no field is blank.
- Repeat the repair step and confirm the row does not change.

Code Reconstruction

The production verifier is:

```
production/formal-papers/CQE-paper-04/verify_boundary_repair.py
```

It verifies:

1. The two Paper 02 correction states are consumed.
2. The Paper 03 coordinates are preserved.
3. Each repaired row has all required fields.
4. Repaired rows are constraints, not proofs.
5. Repair is idempotent.
6. Untyped failures are rejected.

Validation and Hidden-Guess Layer

Useful hidden-guess prompts:

```
What fields must a failed join contain before it is repaired?
Does a repaired boundary row count as proof?
Which Paper 02 states produce boundary repair rows?
What happens to an untyped failure?
```

Expected answers:

```
state, coordinate, reason, status, next legal route, source, target
no, it is a constraint
(0,1,0) and (1,1,0)
it is rejected
```

Open Obligations

- Wire `verify\_boundary\_repair` into `cqe\_engine.formal`.
- Connect boundary constraints to Paper 05 path carriers.
- Promote a shared obligation-ledger schema for all later papers.
- Add a domain example, such as civil crack repair or inventory exception routing, after the formal schema is stable.

Conclusion

Boundary repair is the operation that keeps the corpus honest after failure. It does not turn failed joins into proven claims. It turns them into typed, idempotent, replayable constraints that later papers can legally consume.